

# Custody you cannot quietly rewrite

What the system is, why each piece is shaped the way it is, and what it refuses to claim. Written against the code as it actually stands, hardware and software together.

## 1 The problem is custody, not cryptography

---

Question papers do not usually leak because someone broke the encryption. They leak because somebody who was allowed to hold them handed them to somebody else, and no record exists that could say when.

At Hazaribagh the principal was *authorised* to be in the strong room. There was no lock to pick and no cipher to break. What was missing was an account of who had the package, when, and what they did with it — and by the time anyone asked, the only available answer was a register that could be written up afterwards to say whatever was convenient.

That reframes what a system like this is for. Encrypting the paper is easy and largely beside the point: the people who leak it are the people holding the key. What is hard is producing a record of custody that cannot be edited after the fact by anybody, including the people running the system.

### THE DESIGN QUESTION

Not "how do we stop a leak", which nothing can promise. It is: when a leak happens, can we say with evidence who held the package at every moment, and can that evidence survive the people who would most like to change it?

## 2 One idea, applied everywhere

---

Everything is an append-only sequence of signed facts. State is not stored; it is derived by replaying the sequence.

The consequence people miss is what it does to *absence*. If the current state were a row in a table, deleting an inconvenient step would leave a plausible-looking table. When the state is a replay of events, removing a step leaves a hole — the chain no longer recomputes — and the hole is itself the finding.

Three rules follow, and they explain most of the code:

- **The ledger never refuses to record something that happened.** It rejects only what it cannot *authenticate*: an unknown device, a bad signature, a malformed body. It does not reject events that violate policy — an out-of-order handoff, an unlock outside its window. Refusing to record a violation does not prevent it; it only removes the evidence, recreating the exact invisible gap the system exists to eliminate.
- **Record evidence, not verdicts.** "187" and "41 seconds apart" and "312 m from the centre" can be re-examined years later. "Passed" cannot.
- **A check nobody ran must never look like a check that passed.** Unevaluated checks say so, in those words.

### WHY APPEND-ONLY IS ENFORCED BY THE DATABASE

The application role has no `UPDATE` or `DELETE` grant on the event table. The service refuses to boot if it detects a connection that holds one. The guarantee is a database privilege, not a promise made by application code — code can be changed by whoever deploys it.

### 3 How the chain actually works

---

Two hashes, because they answer two different questions.

```
bodyHash = SHA256( JCS(body) )           ← what the device signs
chainHash = SHA256( prevChainHash || bodyHash ) ← what fixes it in sequence
```

A device cannot know `prevChainHash` when it signs: the chain is global and the device may have been offline for hours. So the signature covers only the body, and the ledger binds that body to a position when it accepts it.

Anyone can then verify two independent things:

- that the authoring device really produced this content — Ed25519 over the body;
- that the ledger has not reordered, inserted or removed anything since — recompute the chain.

### Why RFC 8785, and why it bites

Signing `JSON.stringify` output would be a bug waiting for a deployment. Stringify preserves *insertion* order, so a body that round-trips through Postgres `jsonb`, a queue, or a client-side clone can come back with its keys reordered — same object, different bytes, and every signature over it silently stops verifying. RFC 8785 (JCS) sorts keys and pins number formatting, so the bytes are a function of the value alone.

Two rules fall out of that, and both are load-bearing:

- **Optional fields are omitted, never null.** `{"a":null}` and `{}` are different byte strings. A client that helpfully fills blanks with null produces a body nobody can verify. The signing helper throws on any null before it signs.
- **Timestamps are pinned to millisecond precision, always Z.** `04:30:00Z` and `04:30:00.000Z` are the same instant and different bytes, and only one of them is the one that was signed.

#### WHERE THIS COST US

The ESP32 has no JCS library, so the firmware emits bodies already in canonical form — keys in sorted order, integers only, no nulls, no `geo` — and signs those exact bytes. A runtime guard aborts the build of any body whose keys go out of order, because the alternative is a device that looks healthy and whose every record is rejected in the field with no console attached.

## **Merkle anchors**

Each day's events are hashed into a Merkle tree and the root is published. Anyone holding one event can be given a short inclusion proof against that day's root without being given the rest of the day. The root is the transparency surface: it is public and unauthenticated on purpose.

## 4 What gets recorded

---

Twenty-one event kinds, each with its own typed payload. The discriminated union means `body.payload` narrows from `body.kind` with no casts anywhere in the services.

| GROUP             | KINDS                                                                             |
|-------------------|-----------------------------------------------------------------------------------|
| Package lifecycle | PACKAGE_SEALED · SEAL_APPLIED · HANDOFF · SCAN_OBSERVED                           |
| Access decisions  | ACCESS_REQUESTED · ACCESS_GRANTED · ACCESS_DENIED · OVERRIDE_USED · SEAL_MISMATCH |
| Room telemetry    | MONITOR_HEARTBEAT · MONITOR_SILENT · ROOM_ENTRY                                   |
| Witness station   | WITNESS_ASSERTED · WITNESS_CEREMONY · WITNESS_FRAME                               |
| Key custody       | SHARE_RELEASED · FALLBACK_INVOKED                                                 |
| Printing          | PRINT_STARTED · PRINT_COMPLETED · KEY_DESTROYED                                   |
| Catch-all         | EXCEPTION_RAISED                                                                  |

### Two kinds of asymmetry worth noticing

**Service-only kinds.** A field device may not emit `ACCESS_GRANTED`, `MONITOR_SILENT`, or `SHARE_RELEASED`. A device that could declare itself silent could also decline to; a device that could grant its own access would not be asking.

**Co-signed kinds.** `HANDOFF`, `OVERRIDE_USED` and `FALLBACK_INVOKED` require a second signature from a *different* enrolled device. Two signatures from one device is one person tapping twice, and the whole point of a two-person rule is that it is two people.

### Flags, not refusals

An accepted event can carry advisory flags — clock skew, a missing position fix, a body backdated beyond the offline window. None of them is grounds to refuse the record. They are observations attached to a fact that happened.

## 5 The key, split four ways

---

The content key is split with Shamir into four shares, any three of which reconstruct it.

| SHARE | HELD BY            | RELEASED WHEN                                  |
|-------|--------------------|------------------------------------------------|
| S1    | the authority      | Passphrase-wrapped on our infrastructure       |
| S2    | a drand time lock  | Automatically, once the beacon round publishes |
| S3    | the superintendent | On their authentication at the ceremony        |
| S4    | the observer       | On theirs                                      |

Three of four means no single holder can open a package and no single holder can prevent one. The time lock is what makes "not before T" a property of physics rather than of our access control: before the drand round publishes, S2 does not exist to be leaked, subpoenaed or coerced out of anybody.

### A SHARP EDGE, BLUNTED DELIBERATELY

The Shamir library does not verify reconstruction. Feed it three shares where one is corrupted and it returns a wrong secret with no error — in this setting, an AEAD failure minutes before an exam with no indication of *which* holder supplied bad material. So every share carries a commitment and reconstruction is verified against a commitment to the secret. A bad share can be named.

### AND ONE WE DID NOT BLUNT

Without an HSM, S1 is a passphrase-wrapped blob on our infrastructure. An attacker who fully owns our servers *and* obtains one more share can open a package early. The threshold split means we are not a single point of failure — but this is not HSM-grade custody, and it should not be described as such.

## Custody keys are a different thing

Distinct from the content key: a stage-scoped credential valid for one six-hour epoch. Only its SHA-256 is stored, so a database dump does not yield working keys, and losing one means rotating rather than recovering. That is the intended cost of a credential that cannot be reconstructed from a database.

## 6 The access engine

---

Deny by default. Twenty-one checks, all of them evaluated, none short-circuited.

Never stopping at the first failure costs a few comparisons and buys a complete record. An attempt that trips four checks is a materially different event from one that trips a clock skew, and there is no second chance to observe an attempt that already happened.

| #     | CHECK                                                       | ESTABLISHES                                                     |
|-------|-------------------------------------------------------------|-----------------------------------------------------------------|
| 1–4   | key_presented · key_valid · key_in_window · key_stage_match | A custody key for this stage and this epoch was presented       |
| 5–6   | device_enrolled · device_binding                            | The device is enrolled, unrevoked, and belongs to this centre   |
| 7–8   | roster_membership · role_permitted                          | The named person is on this centre's roster in a permitted role |
| 9–10  | geofence · geo_accuracy                                     | The device is at the centre                                     |
| 11–12 | custody_window · clock_skew                                 | This is a moment the package may be handled                     |
| 13    | seal_serial                                                 | The serial read matches the one registered                      |
| 14–15 | package_state · exam_active                                 | The package can be opened and the exam is not suspended         |
| 16–17 | biometric_primary · biometric_secondary                     | Two different enrolled templates matched                        |
| 18    | two_person_copresence                                       | Both assertions fell inside a short window                      |
| 19    | occupancy_corroborated                                      | The room monitor's independent count agrees                     |
| 20    | seal_lock_intact                                            | <i>Not evaluated — the seal lock is not built</i>               |
| 21    | witness_capture                                             | A photograph was committed by hash at the time                  |

## **Three decisions inside the engine worth knowing**

**A denial is HTTP 200.** A refusal is a successful evaluation that produced "no". A client treating it as a transport error would retry it as though the request had failed to arrive.

**The attempt is written before the answer is returned.** There is no ordering in which a caller learns the outcome without the attempt being on record. A client that crashes on receipt of a denial has still left evidence.

**Hardware checks bind only where hardware exists.** A centre that has never produced a witness event does not have a station, and refusing its unlock for the absence of evidence no device was installed to produce would be a denial about our procurement rather than about the exam. The checks are recorded either way; they contribute deny reasons only once a centre has demonstrably been fitted.

## **Fixed devices are located by their binding**

The geofence exists to prove the requesting device is at the centre. For a phone that means a position fix. For hardware bolted to the wall it does not: a station is placed at accreditation and bound to its centre at enrolment, and that binding is a stronger statement about where it is than a radio fix. Demanding a fix from a device with no GNSS could only ever be satisfied by inventing a number. So a bound station passes on the binding, and the evidence says so in those words — while a phone still has to produce a fix.



# 7 The hardware

Three devices were designed. Two exist.

| DEVICE       | JOB                                                              | STATE                                        |
|--------------|------------------------------------------------------------------|----------------------------------------------|
| Room monitor | Who was in the strong room, continuously, without imaging anyone | Firmware written; sensors not obtained       |
| Witness node | Who opened the bundle, at the moment of opening                  | <b>Built and running</b>                     |
| Seal lock    | Whether the bundle was physically opened                         | Not built — check 20 says so on every unlock |

## Ed25519, not HMAC

The hardware spec said to HMAC each record. Nothing in the software has ever accepted an HMAC: the only entrance to the chain verifies an Ed25519 signature over the canonical body. An HMAC ingest would have needed a second, monitor-shaped entrance with its own verification path — and a second entrance to an append-only chain is exactly what you do not want. The firmware signs with Ed25519, so an ESP32 is authenticated by the same code that authenticates a field phone.

## Two firmware rules that carry most of the value

- **A heartbeat every 30 seconds.** The absence of one is itself a ledger event. Unplugging the device is not a way to go dark; it is a way to raise an alarm. `MONITOR_SILENT` is service-only, so a watchdog derives it — the device cannot report its own silence.
- **Write to storage before transmitting.** Cutting the power or jamming the Wi-Fi delays the record; it does not erase it.

### WHERE THE BUILT DEVICE FALLS SHORT, AND SAYS SO

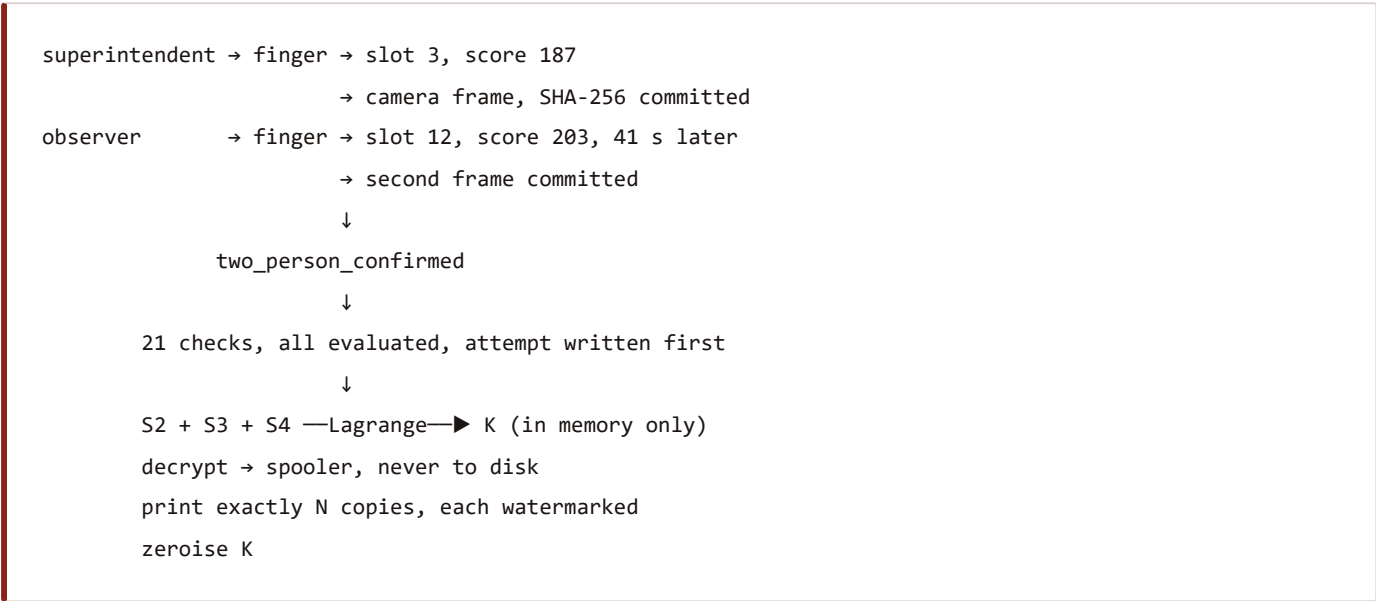
No card is fitted, so records buffer in RAM and do not survive a power cut. No camera is on the board, so every assertion it signs carries a zero frame hash. Both conditions are announced in the ledger on every boot rather than silently absorbed — a device running degraded must be distinguishable in the record from one running properly.

## The fingerprint never leaves the module

The reader enrolls and matches entirely on its own flash and returns two things: a slot id and a score. The ledger records `"slot 3 matched, score 187"`. It has never held a fingerprint image or a template, so a breach of this database cannot leak a biometric that was never in it.

Who slot 3 *is* lives in a separate register — reference data, not chain data. That separation is deliberate: a mapping recorded against the wrong person has to be correctable, and correcting a signed fact is the one thing the chain must never allow. Retired mappings are kept, because an assertion signed last month refers to whoever held the slot then.

## 8 The unlock ceremony, end to end



### The privacy decision, made consciously

The original spec chose a non-imaging presence sensor precisely so there would be no camera anywhere. Adding one reverses a deliberate choice, and the resolution is **scope**, enforced in code rather than promised in a document: the camera fires only when a fingerprint match succeeds. Never on a timer, never on motion, never on a request from the network.

The result is a handful of frames per exam of two officials performing one duty, not hours of a school corridor. Under the DPDP Act that is the difference between a narrow, defensible purpose and a surveillance system.

### The image never enters the ledger

Only `SHA256(jpeg)` is committed. Same pattern as the custody key: commit the hash, hold the artefact elsewhere. The chain proves the image is the one captured at that moment; the chain does not become a photo archive; and losing the image later does not break the chain — it only means that one commitment can no longer be checked.

### Three ways the ceremony does not complete

Each is recorded rather than discarded.

| WHAT HAPPENED                   | RECORDED AS       |
|---------------------------------|-------------------|
| The same finger presented twice | same_finger_twice |
|                                 |                   |

| WHAT HAPPENED                     | RECORDED AS        |
|-----------------------------------|--------------------|
| The second official never arrived | window_expired     |
| A finger matched nothing enrolled | biometric_no_match |

One person tapping twice is not two people — the same principle the ledger enforces when it refuses a co-signature from the signing device itself.

## What each layer contributes to one question

"Who opened package JPR-002, when, and was anyone else present?"

| LAYER        | ITS ANSWER                                                   | FAKEABLE ALONE?                         |
|--------------|--------------------------------------------------------------|-----------------------------------------|
| Ledger       | Device D signed an unlock at 08:52:14, chained at entry #190 | Not without breaking every later hash   |
| Custody key  | A key valid only for that stage and epoch was presented      | Not after the epoch passes              |
| Fingerprint  | Templates 3 and 12 matched, 41 s apart                       | Yes — optical readers are spoofable     |
| Camera       | Two frames, hashes committed at that instant                 | Only by substituting a face, not a hash |
| Room monitor | Door opened 08:50, at least 2 present                        | Yes — flash is readable                 |
| Seal lock    | <i>not built</i>                                             | —                                       |

This is the real argument for the hardware. Individually every physical sensor is defeatable. Together they are independent witnesses that must all be corrupted *consistently* — and the one layer that cannot be quietly rewritten records what each of them said at the time.

## 9 Honest limits

---

Say these before you are asked. Someone who finds one you did not mention will assume there are others you are hiding.

### OPTICAL READERS ARE SPOOFABLE

A lifted print and gelatin will defeat one. These records establish that a body was present, not that the right body was. Optical in the prototype, capacitive in deployment, and the camera frame is the cross-check.

### ESP32 FLASH IS READABLE

Physical access yields the per-device signing key. These devices are tamper-*evident*, not tamper-proof. Their value is the heartbeat gap and the anomaly log, not cryptographic assurance.

### THE TWO-PERSON RULE IS ENFORCED ON TEMPLATES

The system checks that two *different* templates matched. If one person enrolls two fingers under two names, the rule is defeated. That is an enrolment-process failure rather than a system bug — supervised enrolment and the camera frame are the controls — but it is a real gap and worth stating.

### ATTESTATION DOES NOT EXIST YET

Device enrolment accepts an attestation blob and does not verify it. The browser's signing key sits in local storage, not a TPM. Both are recorded as known gaps, and both mean enrolment trusts whoever can reach the endpoint — which is why it must stay behind a gateway that has not been built.

### DIGITAL DELIVERY DOES NOT SCALE TO NEET

Local printing is what collapses the exposure window, so its physical limits bound the whole approach. Roughly 5,000 sides per centre per hour is the ceiling. A large NEET-style centre needs fifteen printers to make the window. The scheduler refuses to place a centre in digital mode that cannot physically finish, and that refusal is a feature.

---

And the one that matters most: at Hazaribagh the principal was *authorised* to be in that room. Biometrics and occupancy sensing are detective controls against unauthorised entry and close to useless against authorised betrayal. Build them; do not let them be the story. The story is the ledger.

## 10 Running it

---

Postgres is the only external service. No Docker, no cloud account, no paid dependency.

```
DATABASE_URL=postgres://mohar_app:...@localhost:5432/mohar \  
  pnpm --filter @mohar/ledger start      # port 8081  
  
pnpm --filter @mohar/control-room dev    # port 5173  
  
tools\monitor-watchdog\run.cmd          # derives MONITOR_SILENT
```

Migrations run as a separate migrator role, never as the app role. The app role deliberately lacks the privileges to alter the ledger, and that absence *is* the append-only guarantee.

### Two things that will catch you

- **Restart the ledger after changing contracts.** It validates against its compiled schema, so a running instance rejects new event kinds with `schema_invalid` until restarted.
- **Open the control room on `localhost`.** Browsers grant camera access on localhost or HTTPS only, and an HTTPS page cannot call the station over plain HTTP.

### Reading a refusal

The refusals are almost always correct. The seeded data cannot be granted and that is deliberate: every seeded custody window has closed, and several centres were seeded specifically to demonstrate refusal. An engine that granted them would be an engine that had stopped checking.

---

Mohar — study guide, written against the code as it stands. Companion to the hardware build guide in `docs/12` and the roadmap in `docs/learn`. Software described here is implemented except where marked otherwise; the seal lock and the gateway are named as not built wherever they appear.